Sentrais Blueprint Generator

It accepts a JSON spec and builds the full Figma page structure for the master design blueprint

You can reuse the same generator for every ecosystem by swapping the JSON input

1) Create the JSON Spec

Create a file called sentrais_master_blueprint.json:

{

"fileName": "Sentrais_Master_Ecosystem_Blueprint_v1.0",

"pages": [

```
{
  "name": "00 – FOUNDATION",
  "sections": [
    { "name": "Design North Star", "frames": ["Design North
Star"] },
    { "name": "Ecosystem Mental Model", "frames": ["Mental Model",
"Operating Modes", "Role Responsibility"] },
    { "name": "Navigation Rules", "frames": ["Navigation Rules",
"Role Gating Rules", "Drill-down Rules"] }
  ]
},
{
  "name": "01 – DESIGN SYSTEM",
  "sections": [
    { "name": "Color Tokens", "frames": ["Color Tokens", "Status
Colors", "System Colors"] },
    { "name": "Typography", "frames": ["Typography Scale", "Numeric
Styles", "Metadata Styles"] },
```

```
    { "name": "Spacing & Layout", "frames": ["Grid & Spacing",
"Containers", "Padding Rules"] },
    { "name": "Icons", "frames": ["Nav Icons", "Status Icons",
"System Icons"] },
    { "name": "Motion Guidelines", "frames": ["Motion Principles",
"Transitions", "Drawer Behavior"] }
  ]
},
{
  "name": "02 – CORE COMPONENTS",
  "sections": [
    { "name": "Navigation", "frames": ["Primary Left Rail",
"Collapsed Rail", "Context Top Bar", "Breadcrumb / Context Pill"] },
    { "name": "Status & Signals", "frames": ["Status Badge",
"Readiness Ring", "Risk Indicator", "Countdown Timer"] },
    { "name": "Cards", "frames": ["KPI Card", "System Card", "Task
Card", "Risk Card", "Game Card"] },
    { "name": "Interaction", "frames": ["Buttons", "Dropdowns",
"Tabs", "Toggles"] },
    { "name": "Containers", "frames": ["Timeline", "Tables",
"Lists", "Drawers"] }
  ]
},
{
  "name": "03 – MASTER NAVIGATION",
  "sections": [
    { "name": "Nav Variants", "frames": ["Primary Nav – Default",
"Primary Nav – Collapsed"] },
    { "name": "Role Variants", "frames": ["Executive Nav", "Ops /
GDA Nav", "Club Nav", "Vendor Nav", "Admin Nav"] },
    { "name": "Context Bar Variants", "frames": ["Season Context",
"Week Context", "Game Day Context", "Crisis Mode Context"] }
  ]
},
{
  "name": "04 – MODE BLUEPRINTS",
  "sections": [
```

```
    { "name": "Core Modes", "frames": ["Command Mode", "Operations
Mode", "Systems Mode", "Risk Mode", "Review Mode", "Governance
Mode"] }
  ]
},
{
  "name": "05 – EVERGAME MASTER EXPERIENCE (NFL)",
  "sections": [
    { "name": "League Command", "frames": ["League Command
Overview", "Season Snapshot", "Week at a Glance", "Game Risk
Summary"] },
    { "name": "Pre-Season Readiness", "frames": ["Readiness
Overview", "Club Readiness Comparison", "Certification Status"] },
    { "name": "In-Season Operations", "frames": ["In-Season
Overview", "Week View", "Game View – Tabs Layout"] },
    { "name": "Post-Game Review", "frames": ["Post-Game Summary",
"Findings", "Evidence Ledger", "Improvement Actions"] },
    { "name": "Credentials & Compliance", "frames": ["Certifications
Dashboard", "Renewal Detail", "Access & Roles"] },
    { "name": "Risk & Integrity", "frames": ["Risk Matrix", "Drivers
& Mitigations", "Scenario View"] },
    { "name": "Intelligence Reports", "frames": ["Reports Library",
"Executive Brief", "Audit Pack"] }
  ]
},
{
  "name": "06 – WEEK-OF-GAME GOLDEN PATH",
  "sections": [
    { "name": "Golden Path", "frames": ["Week Selected", "T-7
Baseline", "T-3 Execution", "T-24 Live Ops", "Kickoff Live",
"Incident Escalation", "Post-Game Review", "Certification Update"] }
  ]
},
{
  "name": "07 – ROLE-SPECIFIC FLOWS",
  "sections": [
    { "name": "Executive", "frames": ["Exec Landing", "Exec Week
Drill-down", "Exec Report Export"] },
```

```
    { "name": "Ops / GDA", "frames": ["Ops Weekly Execution", "Ops
Milestone Run", "Ops Incident Resolution"] },
    { "name": "Club", "frames": ["Club My Tasks", "Club Evidence
Upload", "Club Certification Check"] },
    { "name": "Vendor", "frames": ["Vendor Assigned Systems",
"Vendor Task Completion", "Vendor Proof Submission"] }
  ]
},
{
  "name": "08 – STATES & EDGE CASES",
  "sections": [
    { "name": "States", "frames": ["Empty State", "Loading",
"Error", "Escalation", "Locked / Restricted", "Degraded Mode"] }
  ]
},
{
  "name": "09 – HANDOFF & IMPLEMENTATION",
  "sections": [
    { "name": "Engineering Handoff", "frames": ["Component → Code
Mapping", "Route Mapping", "Data Binding Notes", "Role Gating
Logic", "API Touchpoints"] }
  ]
}

]
```

}

2) Figma Plugin Code

Create a plugin with two files:

code.ts

```
// Sentrais Blueprint Generator (Figma Plugin)
// Generates pages, sections, and frames from a JSON spec.
type Blueprint = {
 fileName?: string;
 pages: Array<{
```

```
name: string;
sections: Array<{
  name: string;
  frames: string[];
}>;

}>;
```

```
    };
    const FRAME_W = 1200;
    const FRAME_H = 700;
    const PADDING_X = 80;
    const PADDING_Y = 80;
    const GAP_X = 60;
    const GAP_Y = 60;
    function createSectionHeader(page: PageNode, name: string, y: number) {
     const text = figma.createText();
     text.characters = name;
     text.fontSize = 28;
     text.fontName = { family: "Inter", style: "Bold" };
     text.x = PADDING_X;
     text.y = y;
     page.appendChild(text);
     return text;
    }
    function createFrame(page: PageNode, name: string, x: number, y: number) {
     const frame = figma.createFrame();
     frame.name = name;
     frame.resize(FRAME_W, FRAME_H);
```

```
    frame.x = x;
    frame.y = y;
    frame.fills = [{ type: "SOLID", color: { r: 0.98, g: 0.98, b: 0.99 } }];
    frame.strokes = [{ type: "SOLID", color: { r: 0.85, g: 0.86, b: 0.9 } }];
    frame.strokeWeight = 1;
    const label = figma.createText();
    label.characters = name;
    label.fontSize = 20;
    label.fontName = { family: "Inter", style: "Medium" };
    label.x = 24;
    label.y = 24;
    frame.appendChild(label);
    page.appendChild(frame);
    return frame;
  }
  async function run(spec: Blueprint) {
    await figma.loadFontAsync({ family: "Inter", style: "Bold" });
    await figma.loadFontAsync({ family: "Inter", style: "Medium" });
    // Optional: rename file (Figma doesn't allow programmatic file rename, but we keep name in UI)
    const createdNodes: SceneNode[] = [];
    for (const p of spec.pages) {
const page = figma.createPage();
page.name = p.name;

let cursorY = PADDING_Y;
```

```
for (const section of p.sections) {
  const header = createSectionHeader(page, section.name, cursorY);
  createdNodes.push(header);

  cursorY += 60;

  // Layout frames in 2 columns
  let col = 0;
  let row = 0;

  for (let i = 0; i < section.frames.length; i++) {
    col = i % 2;
    row = Math.floor(i / 2);

    const x = PADDING_X + col * (FRAME_W + GAP_X);
    const y = cursorY + row * (FRAME_H + GAP_Y);

    const frame = createFrame(page, section.frames[i], x, y);
    createdNodes.push(frame);
  }

  // Advance cursor after the section grid
  const rows = Math.ceil(section.frames.length / 2);
  cursorY = cursorY + rows * (FRAME_H + GAP_Y) + 80;
}

}
```

```
figma.currentPage = figma.root.children[0] as PageNode;
figma.viewport.scrollAndZoomIntoView(createdNodes.slice(0, 6));
figma.notify("Sentrais Blueprint generated.");
figma.closePlugin();
}
figma.showUI(__html__, { width: 420, height: 520 });
figma.ui.onmessage = async (msg) => {
```

if (msg.type === "generate"){

```
try {
  const spec = JSON.parse(msg.specText) as Blueprint;
  await run(spec);
} catch (e: any) {
  figma.notify(`Error: ${e?.message || "Invalid JSON"}`);
}
```

}

};

ui.html

<!doctype html>

```
<meta charset="utf-8" />
<style>
  body { font-family: Inter, system-ui, -apple-system, Segoe UI,
Roboto, Arial; margin: 16px; }
  textarea { width: 100%; height: 340px; font-family: ui-monospace,
SFMono-Regular, Menlo, Monaco, Consolas, monospace; font-size: 12px;
}
  button { width: 100%; padding: 12px; font-weight: 600; margin-top:
12px; }
  .hint { color: #555; font-size: 12px; margin-top: 8px; line-
height: 1.4; }
</style>
```

```
<h3>Sentrais Blueprint Generator</h3>
<div class="hint">Paste your blueprint JSON spec below. The plugin
will generate pages, section headers, and starter frames.</div>
```

```
<textarea id="spec" placeholder="Paste JSON spec here"></textarea>
<button id="go">Generate Blueprint</button>

<script>
  document.getElementById('go').onclick = () => {
    const specText = document.getElementById('spec').value;
    parent.postMessage({ pluginMessage: { type: 'generate', specText
} }, '*');
  };
</script>
```

3) How You Use It

In Figma: Plugins → Development → New Plugin

Choose “Link existing plugin” and point to your folder containing code.ts and ui.html

Run the plugin

Paste the JSON spec

Click Generate Blueprint

You will instantly get the full master ecosystem page structure with starter frames.